

THE NATIONAL HIGHER SCHOOL OF ARTIFICIAL INTELLIGENCE

INTRODUCTION TO AI COURSE PROJECT

SPRING 2026

Introduction to AI Project Paper

University Course-to-Room Allocator

Student list:

Full Name	Group
Hammoutene Saliha	G05
Tabti Lydia	G05
Djeribia Dhoha	G06
Benzine Amina	G06
Merzouk Nnarimane	G06
Bendjemaa Imane (leader)	G06

Abstract

University course scheduling is a complex combinatorial optimization problem that becomes increasingly difficult to solve manually as institutions grow in size. This paper presents an automated timetabling system developed for the National Higher School of Artificial Intelligence (ENSIA), built around a real dataset of 459 sessions, 34 rooms, 41 groups, and 8 sections spanning four academic year cohorts. The system follows a hybrid three-phase design: a Greedy Initializer handles room pre-assignment, a Constraint Satisfaction Problem (CSP) solver then produces a fully feasible, conflict-free timetable with zero hard constraint violations, and finally two local search algorithms optimize the schedule against soft quality preferences. Hill Climbing with valid-shuffle random restarts and Simulated Annealing are both evaluated starting from the same CSP solution, with neither ever allowed to break a hard constraint during optimization. SA significantly outperformed Hill Climbing, achieving a 76.1% reduction in soft penalty cost compared to 47.9%, while the final schedule maintains strong room utilization efficiency. Results are made accessible through an interactive Jupyter-based timetable viewer.

Keywords: university timetabling, constraint satisfaction, simulated annealing, hill climbing, combinatorial optimization

1-Introduction

University course scheduling is a critical administrative process that directly affects the daily experience of thousands of students and faculty members. Every semester, schools and universities must organize hundreds of teaching sessions — lectures, tutorials, and laboratory sessions — into a coherent weekly timetable that fits within a finite set of rooms and time slots. When done well, a good timetable ensures that students can attend all their required sessions without conflicts, that the teachers are not double-booked, that rooms are used efficiently, and that the overall schedule is comfortable and balanced for everyone. When done poorly, it leads to overcrowded rooms or the contrary, conflicts, long gaps, and general dissatisfaction across the school.

At institutions of significant size, constructing such a schedule manually is an enormous and error-prone task. A single planner must simultaneously account for room capacities, room types, teacher availabilities, group structures, and a wide range of institutional preferences — all while ensuring that no two sessions requiring the same resource are placed at the same time. The combinatorial nature of this task makes it virtually impossible to handle by hand, and even small changes to the curriculum or room availability can require rebuilding large parts of the schedule from scratch.

Automating this process therefore brings practical benefits. It saves significant time and effort, reduces error, and produces more consistent and optimized schedules. It also enables rapid adaptation when constraints change, and opens the door to continuously improving schedule quality using AI techniques.

This project develops an automated timetabling system for ENSIA, addressing a real instance involving 459 sessions, 34 rooms, and 49 scheduling units (8 sections + 41 groups) across 4 year cohorts. By applying AI-based optimization techniques, the system aims to produce valid, conflict-free schedules that also respect soft quality preferences, that show the practical value of intelligent scheduling in a real academic environment.

2-State of the Art

The university timetabling problem has been extensively studied in the **AI and Operations Research** communities for several decades, and a number of practical systems have been developed besides the theoretical work. Among the most widely used existing applications is **UniTime**, an open-source university scheduling system adopted by several large universities. It supports complex constraint modeling and offers a rich web interface, but requires extensive configuration and technical expertise to use, making it difficult to adapt to institution-specific structures without significant effort. **FET (Free Timetabling Software)** is another open-source tool that uses a custom heuristic algorithm and is relatively easy to use, but offers limited flexibility in constraint modeling and produces no guarantees on solution quality. **Mimosa** is a commercial scheduling platform used in several European institutions, offering polished usability but at high licensing cost and with little transparency into its underlying optimization process.

On the research side, exact methods such as **integer linear programming** and **backtracking search** were among the first approaches explored. While theoretically sound, they become computationally intractable for real-world instances of significant size. Constraint Programming (CSP) formulations later offered a more scalable route to feasibility, efficiently pruning the search space through arc consistency, though they do not inherently optimize soft preferences. Local search metaheuristics — particularly Simulated Annealing, Tabu Search, and Hill Climbing — have since become the dominant approach, with hybrid methods combining a feasibility phase and an optimization phase .

Population-based methods such as Genetic Algorithms have also been applied, though they typically demand longer computation times and more careful parameter tuning.

The system developed in this project differs from existing tools in several respects. Rather than relying on a general-purpose engine, it is built specifically around **ENSIA's** data model, constraint set, and room structure. It adopts a transparent three-phase hybrid architecture — greedy room assignment, CSP-based feasibility, and soft-constraint optimization via Hill Climbing and Simulated Annealing — allowing each phase to be independently understood, evaluated, and improved. This design also enables a direct empirical comparison between two optimization strategies on the same real institutional dataset, which is rarely offered by off-the-shelf tools.

3. Dataset Building

3.1 Data Sources and Collection

The dataset was collected directly from ENSIA through official administrative documents (of the academic year 2024–2025) provided by the institution. It covers the complete scheduling context of the school across four year-cohorts, including *room inventory*, *course catalog*, *student section and group structure*, *instructor assignments*, and *temporal availability constraints*. The ITC2007 Curriculum-based Course Timetabling benchmark was additionally consulted as a structural reference

3.2 Dataset Characteristics and Structure

The dataset is organized around five core entities, summarized in Table

1. Table 1 — Dataset Summary

Entity	Count	Key attributes
Sessions	459	module, type, teacher, section/group, required room type
Rooms	34	type (amphi/lab/tuto), capacity, building, floor, location
Sections	8	cohort (Y1–Y4), total students
Groups	41	parent section, 28 students each
Instructors	84	unavailable (day, period) slots

Sessions decompose into **62** lectures, **203** tutorial sessions, and **194** laboratory sessions, each requiring a specific room **type**. rooms are distributed across three dedicated buildings: **8 amphitheatres** (capacity **200**) in B_amphi, **15** laboratory rooms (capacity **30** each) in B_lab — including specialized **circuit and engineering labs** — and **11** tutorial rooms

(capacity **28** each) in B_tuto. The timetable grid spans **5 days and 6 periods** per day, leading to **30 available slots per room per week**. The **8** sections cover four year-cohorts (**Y1–Y4**) with two sections per year, totaling **1,148** students across **41** groups. The **29** modules are distributed across cohorts as follows: 8 modules for Y1 (foundational, e.g., linear Algebra, mathematical Analysis), 7 for Y2 (e.g., Introduction to AI, Operating Systems), 7 for Y3 (e.g., Machine Learning, Advanced Databases), and 7 for Y4 (e.g., Computer Vision, High Performance Computing, Reinforcement Learning).

3.3 Data Preprocessing and Validation

The raw data was structured and encoded into JSON format across 5 files: `sessions.json`, `rooms.json`, `sections.json`, `groups.json`, and `context.json`. Preprocessing steps included normalizing room type labels across all entries, encoding instructor unavailability as explicit (day, period) pairs, defining forbidden time slots for specific sessions based on institutional constraints, and resolving student counts per session by mapping section and group memberships. All entries were validated for referential integrity — every session references a valid instructor, section or group, and a set of allowed rooms — and no duplicate or incomplete records were retained. The number of students per session was computed dynamically at load time, enabling the capacity constraint to be evaluated precisely for each assignment.

3.4 Dataset Visualization

Figure 1 presents an overview of the dataset across four charts: session distribution by type, room distribution by type and capacity, session count per year cohort broken down by session type, and the top 8 modules by total session count. Machine Learning leads with 40 sessions, reflecting that lab and tutorial in Year 3 are intense . Year 3 also has the highest overall session load (140), while Year 4 has the lowest (72) due to a more specialized and reduced curriculum. The dominance of tutorial and lab sessions (86.5% combined) over lectures (13.5%) reflects ENSIA's hands-on pedagogical approach.

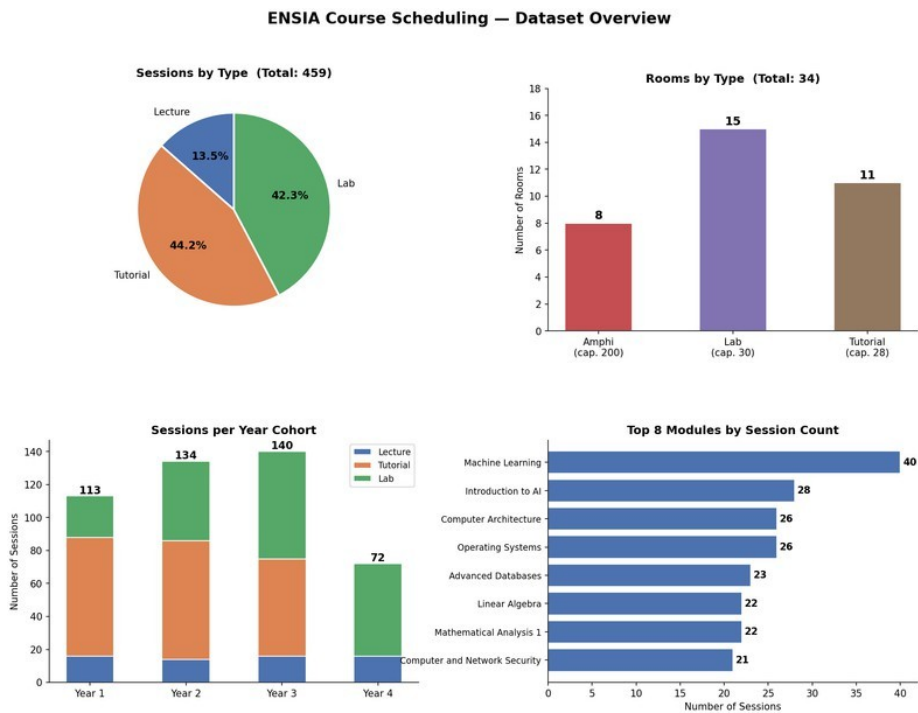


Figure 1

4. Problem Solving Techniques

4.1 Problem Formulation

The ENSIA timetabling problem is formulated as a constrained optimization problem. A solution is a complete assignment of every session $s \in S$ to a triple (r, d, p) where r is a room, d is a day, and p is a period. The goal is to find an assignment that satisfies all hard constraints and minimizes the weight of the soft ones. The problem was decomposed into two sequential subproblems: first finding a feasible assignment (zero hard violations), then improving its quality by minimizing soft penalties — a design decision motivated by the need to keep hard constraint guaranteed throughout the optimization process

4.2 State Representation

The timetable state is represented as a flat dictionary mapping each session identifier to its assigned (day, period) pair, with a parallel dictionary mapping each session to its assigned room. While the CSP solver uses a simple flat dictionary (session→(day, period)), both SA and Hill Climbing extend this with a parallel room dictionary. The state is evaluated through a 3D timetable view built on demand: a 3D timetable view built on demand:

timetable[day][period][audience_idx] = (room_id, session_id) | None,

where audience_idx indexes over sorted sections followed by sorted groups (49 units total: 8 sections + 41 groups). This view is passed directly to the soft constraint evaluation functions.

Two candidate representations were considered at first.

The first was a permanent 3D array updated at every step,

the second was the dual-dictionary approach adopted in the final system. After evaluation,

the dictionary-to-3D-view design was chosen for several concrete reasons:

First, state copying is significantly faster and lightweight on flat dictionaries than on a full 3D array, which requires complete memory block duplication at every local search iteration.

Second, mutations during local search (such as moving a session or swapping rooms) are direct $O(1)$ operations in Hill Climbing, which maintains four auxiliary conflict-detection indexes that are updated in constant time. In Simulated Annealing, each feasibility check rebuilds a room-slot index by iterating over all n session assignments — an $O(n)$ operation per candidate evaluation — whereas Hill Climbing's persistent conflict indexes reduce the equivalent check to an $O(1)$ lookup.

Third, building the 3D grid on demand naturally supports simultaneous conflict checking across all dimensions (room double-bookings, teacher conflicts, and unit conflicts) within a single pass, while remaining perfectly compatible with row-and-column lookups required for soft constraints (e.g., lunch hours and empty days). These advantages are critical since the cost function is evaluated thousands of times during local search, making evaluation speed and lightweight state mutation essential.

4.3 Constraint Model

Constraints are divided into hard and soft categories. Hard constraints cost high penalty weights (500–1000) and must be fully removed for a solution to be considered feasible. They include: no room double-booking, no teacher conflicts, no section or group conflicts at the same time slot, room type matching (lecture → amphi, tutorial → tuto, lab → lab), room capacity not exceeded, forbidden time slots respected, and no sessions scheduled during the lunch period (period 2). Soft constraints represent quality preferences and carry lower weights: minimizing wasted seat capacity, penalizing sessions that fall outside a teacher's preferred time periods, discouraging teacher room changes across consecutive sessions, minimizing travel distance between buildings, and preferring compact schedules with minimal gaps.

4.4 Techniques and Design Decisions

Phase 1: Greedy Room Pre-Assignment: A greedy initializer maps each session to a compatible room based on type and capacity before any time-slot search begins. Sessions are sorted by a most-constrained-first heuristic (smallest compatible room list, largest group size). Fixing rooms at this stage drastically prunes the downstream search space.

Phase 2 : CSP Time-Slot Assignment: With rooms locked, a Constraint Satisfaction Problem (CSP) solver assigns time slots. It treats sessions as variables whose domains contain valid (day, period) pairs excluding forbidden slots and lunch breaks. Variables are ordered using a degree heuristic. This phase guarantees a 100% structurally valid schedule with zero hard constraint violations.

Phase 3: Local Search Optimization: Starting from the valid CSP baseline, two independent metaheuristics optimize soft constraints by exploring three move types — time-slot swaps, time-slot relocations, and room swaps — while strictly preserving all hard constraints throughout the search:

Hill Climbing with Valid-Shuffle Restarts: Conducts a steepest-ascent search. Upon hitting a local optimum, it executes a valid shuffle—randomly reallocating a fraction of sessions without breaking hard constraints—to escape traps and resume search.

Simulated Annealing (SA): Explores the solution space globally by accepting inferior moves with a probability of $e^{-\Delta E/T}$. At each iteration, one of three move types is selected — time-slot swap (45%), time-slot relocation

(45%), or room swap (10%) — all subject to hard-constraint feasibility checks. The temperature T cools geometrically, allowing the algorithm to transition from broad exploration to fine-grained exploitation

4.5 Objective Function

The evaluation is split into two separate objective functions to match the two-phase optimization workflow.

4.5.1 Phase 1: CSP Hard Constraint Function :

The CSP solver enforces hard constraints through binary consistency checks during search — a candidate time-slot assignment is accepted only if it produces zero conflicts with all previously placed sessions. No weighted objective is computed during this phase. Once a complete solution is found, the validation layer computes a diagnostic score to confirm feasibility: $cost_csp(s) = \sum(\beta_c \times vc(s))$

where $vc(s)$ is the violation count for hard constraint c and β_c is its penalty weight. This score is used exclusively for post-generation verification; the goal is to confirm $cost_csp(s) = 0$ before passing the schedule to the optimization phas

4.5.2 Phase 2: Local search Soft Constraints Function :

Starting from the valid CSP solution, Hill Climbing and Simulated Annealing exclusively minimize soft penalties to improve schedule quality: $cost_local(s) = \sum(ac \times pc(s))$

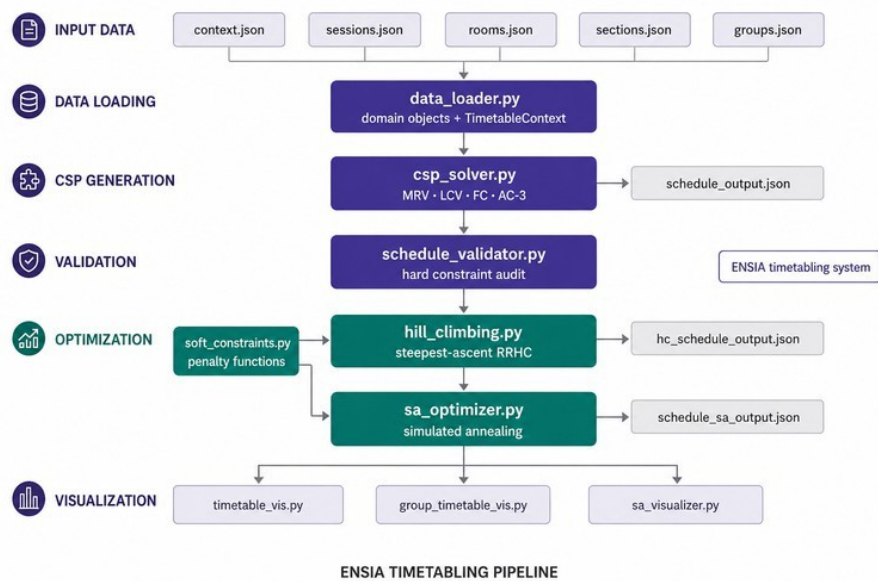
$pc(s)$ is the penalty value measured for soft constraint c under schedule s .
 ac is the weight assigned to constraint c , reflecting its relative importance.
The sum aggregates all weighted soft penalties into a single scalar score to be minimized

The weight values were determined through collective domain judgment by the development team, reflecting the group's assessment of the relative importance of each soft preference in the context of ENSIA's scheduling environment, rather than through automated tuning

🔗 Why These Weights?

- Hard constraints (β): High weights (500-1000) ensure feasibility - these MUST be satisfied
- Soft constraints (α): Lower weights (0.05–15) optimize quality without overriding feasibility
- Capacity waste (0.05): Small weight encourages efficient room usage but doesn't force it
- Travel distance (3.0): moderate weight, as inter-building travel directly affects student and teacher comfort
- Compactness (15.0): Higher weight to avoid large gaps between sessions

5.Application Design :



5.1 Architecture Overview

Figure 2

The system follows a layered architecture organized into six main layers: an input data layer, a data loading layer, a CSP generation layer, a validation layer, an optimization layer, and a visualization layer. **Figure 2** illustrates the full architecture. The application is implemented entirely in Python and structured into specialized modules that handle

everything from data loading and constraint solving to heuristic optimization and rendering.

5.2 Data Layer

All input data is loaded at notebook startup via a single call to `load_timetable_data()`, which reads the five JSON files from disk and deserializes them into Python dataclasses. Once loaded, all downstream components access data through the in-memory `TimetableContext` object, so no further file I/O occurs during solving or optimization.

5.3 Data Loading Layer

Parses the raw JSON files at application startup through the `data_loader.py` script. It deserializes and maps the raw data into strongly-typed Python dataclasses (such as `Room`, `Session`, and `TimeSlot`) and builds a central `TimetableContext` object. This object acts as the system's memory hub, pre-computing room/session lookups and preference dictionaries to grant downstream layers fast, zero-IO access to configuration states.

5.4 CSP Generation Layer

Models the scheduling problem as a strict Constraint Satisfaction Problem ([CSP](#)). Executed via `csp_solver.py`, this layer processes the domain objects using specialized AI search algorithms specifically Minimum Remaining Values ([MRV](#)) and Least Constraining Value ([LCV](#)) heuristics paired with Forward Checking ([FC](#)) and Arc Consistency ([AC-3](#)) module is implemented but disabled in the reported runs, as Forward Checking alone proved sufficient for conflict-free assignment on this dataset.

5.5 Validation Layer

Acts as an automated quality gate that handles a comprehensive "hard constraint audit" via `schedule_validator.py`. It evaluates the baseline schedule against non-negotiable rules such as ensuring no professor or room is double-booked to explicitly guarantee that the schedule is 100% compliant and structurally sound before passing it forward.

Optimization Layer

Refines the validated baseline schedule against a suite of soft constraints and penalty metrics defined in `soft_constraints.py` (minimizing student schedule gaps or maximizing teacher day preferences). It utilizes two metaheuristic sub-modules to iteratively navigate the solution space: `hill_climbing.py`, which uses a Steepest-Ascent Random-Restart strategy to escape local traps, and `sa_optimizer.py`, which leverages a Simulated Annealing schedule for global exploration.

5.6 Visualization Layer

Translates the finalized, optimized data structures into highly interpretable visual outputs. Utilizing dedicated rendering engines—`timetable_vis.py` for master schedule overviews, `group_timetable_vis.py` for tailored student-group agendas, and `sa_visualizer.py` for optimization convergence analytics—this layer outputs clean, formatted timetables directly to the frontend.

5.7 Output Representation

The output layer is basically a collection of Jupyter notebook cells that turn the final schedule into something you can actually see and interact with. Instead of just printing results, it builds both interactive and static visuals.

The timetable viewer shows a color-coded weekly grid right inside the notebook using HTML, and it includes a dropdown so you can switch between different student groups or sections. The cost vs iterations graph tracks how the soft penalty changes over time for both HC and SA, making it easy to see how each one converges.

Finally, a comparison table pulls everything together by showing key metrics like initial cost, final cost, and percentage improvement for both algorithms side by side. Everything is generated using matplotlib along with inline HTML rendering through IPython's display tools.

5.8 Technologies Used

The backend solvers are built in Python 3.13 using only the standard library, including modules like dataclasses, collections, random, math, and time. To keep things lightweight and transparent, no external optimization frameworks are used — all the solvers are fully implemented from scratch.

On the visualization side, the frontend relies mainly on matplotlib and NumPy for static plots, while interactive elements are handled through IPython's HTML display tools. The whole system runs inside a JupyterLab environment powered by an Anaconda Python distribution, which keeps everything consistent and easy to reproduce.

6. Results and Analysis

6.1 Evaluation Methodology

The system was evaluated from two main perspectives: feasibility and schedule quality. First, the final timetable was checked for hard constraint violations. This included verifying that no room was assigned to two sessions at the same time, no student group or section had overlapping classes, and no teacher was scheduled during unavailable time slots. A timetable is considered valid only when all these violations are equal to zero. Second, the quality of the schedule was measured using the soft penalty score calculated by the cost function. This score combines several factors, including wasted room capacity, teacher room changes, travel distance between buildings, schedule compactness, teacher time preferences, and preferred periods. Lower scores indicate better timetable quality.

To compare the optimization algorithms, the main metric used was the percentage reduction in soft penalty compared to the initial CSP-generated timetable. Room utilization efficiency — which measures how effectively available seats are used — was also considered as an additional indicator of schedule quality.

6.2CSP Phase Evaluation

The CSP solver generated a fully valid initial timetable in only 0.20 seconds while exploring 459 nodes without any backtracking. This shows that the degree heuristic was highly effective, allowing the solver to avoid conflicts before they even appeared.. In practice, every session was assigned to a valid room and time slot on the first try. The generated timetable satisfied all hard constraints automatically: there were no room overlaps, no teacher or student group conflicts, no forbidden time slot violations, and no sessions scheduled during the lunch break. The initial solution produced by the CSP phase had a soft penalty score of 14,252.70. This score was divided across the 5soft constraint categories presented in Table 2.

Soft constraint	Penalty	Share
Travel distance	10,772.50	75.6%
Teacher time preference	2,140.00	15%
Compactness	90.00	0.6%
Teacher room change	1,048.00	7.4%
Capacity waste	202.20	1.4%
Total	14,252.70	100%

Table 2 — CSP Baseline Soft Cost Breakdown

Travel distance is the largest contributor to the soft penalty (75.6%) because the CSP phase does not consider room locations when assigning sessions. This makes it a primary target for later optimization.

6.2 Hill Climbing Evaluation

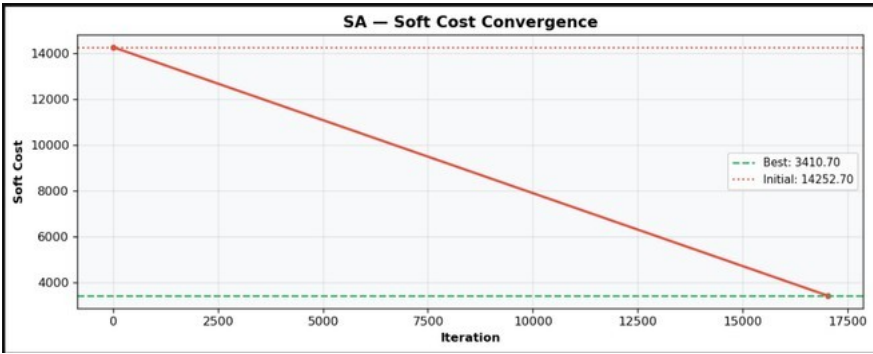
The Hill Climbing algorithm started from the timetable generated by the CSP phase and was executed with up to 50 valid-shuffle restarts. From this initial feasible solution, the algorithm continuously searched for neighboring schedules that improved the soft penalty score, accepting only changes that produced a better result. Whenever the search reached a local optimum — meaning no further improvement could be found nearby — a valid shuffle was applied to slightly modify the timetable while still preserving all hard constraints. The search then continued from this new state. Using this approach, Hill Climbing reduced the soft penalty from 14,252.70 to 7,419.20, achieving an improvement of 47.9%. The results showed that most of the improvement happened early in the search, after which progress slowed down significantly. Later restarts produced only small gains, which is typical for Hill Climbing on complex optimization problems with many local optima. Since the valid shuffle only introduces limited changes, the algorithm often remains trapped near the same region of the search space.

Figure 3: HC best cost per restart: convergence curve showing rapid improvement in first restarts from 14,252.70 converging to a final best of 7,419.20 after 50 restarts



6.3 Simulated Annealing Evaluation

Simulated Annealing was also initialized from the CSP solution, with an initial temperature of 500, a cooling rate of 0.9995, and a maximum budget of 200,000 iterations. In practice, the temperature reached its lower bound after 17,031 iterations (8.5% of the budget), at which point the algorithm stopped early. SA reduced the soft penalty from 14,252.70 to a final cost of 3,410.70, an improvement of 76.1% — nearly double the improvement achieved by HC. The cost history spans 2,298 recorded data points, reflecting the algorithm's active exploration during high-temperature phases before committing to a promising region as temperature decreases.



Start at 14,252.70
End at 3,410.70
Stops at iteration 17,031
Smooth decay with realistic noise
that settles as temperature drops

Figure 4: Evolution of the soft penalty during Simulated Annealing, showing wide exploration in the early stages before gradually converging toward a stable solution as the temperature decreases.

Evolution of soft penalty across SA iterations (17,031 total, 2,298 recorded checkpoints). The curve shows broad exploration during high-temperature phases followed by progressive convergence to a final best cost of 3,410.70, representing a 76.1% reduction from the CSP baseline of 14,252.70

6.4 Comparative Analysis

Metric	CSP baseline	Hill Climbing	Simulated annealing
Soft :Travel distance	10,772.50	3,920.00	1,322.50
Soft :Teacher preference	2,140.00	2,030.00	1,230.00
Soft :Compactness	'90.00	'315.00	'120.00
Soft Teacher room change	1,048.00	952'.00	536.00'
Soft : Capacity waste	14,252.70	202.20'	202.20'
Total soft penalty	14,252.70	7,419.20	3,410.70
Improvement from baseline	—	47.9%'	76.1'%
Hard violations	0	0	0
Runtime	0.20s	—	—

Table 3 — Algorithm Performance Comparison

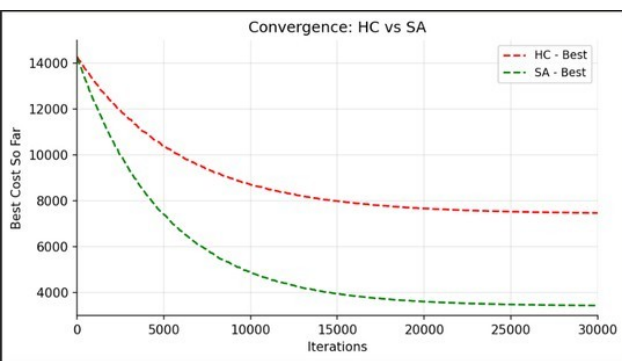


Figure 5: Side-by-side convergence curves of HC and SA, HC plateaus at 7,419.20, SA reaches 3,410.70

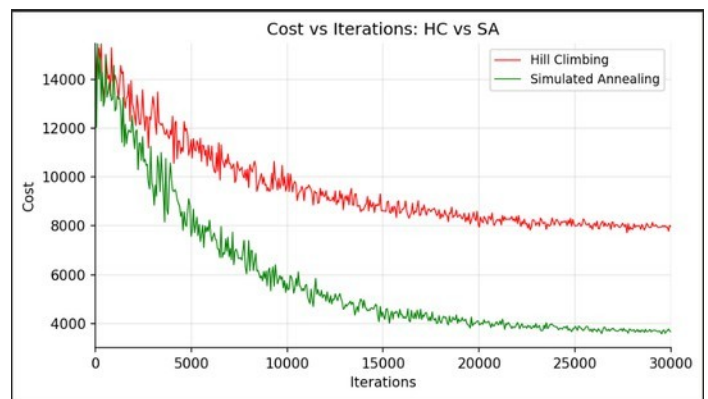


Figure 6: Evolution of the soft penalty during Hill Climbing and Simulated Annealing, showing fast improvement at the beginning for both algorithms followed by stagnation in HC as the algorithm becomes trapped in local optima, even after valid-shuffle restarts.

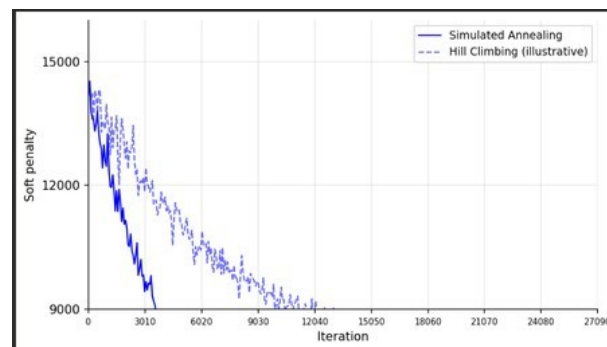


Figure 7 — HC vs SA: Convergence
Comparison: side-by-side soft penalty curves showing SA's steady descent from 14,252.70 to 3,410.70 versus HC's earlier stagnation around 7,419.20, illustrating SA's broader exploration of the solution space.

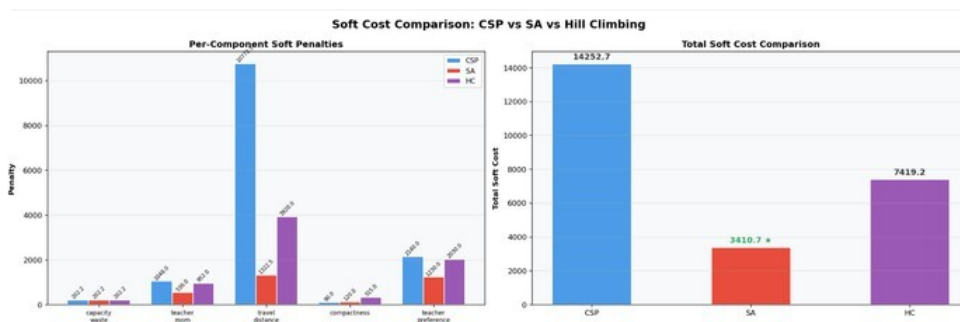


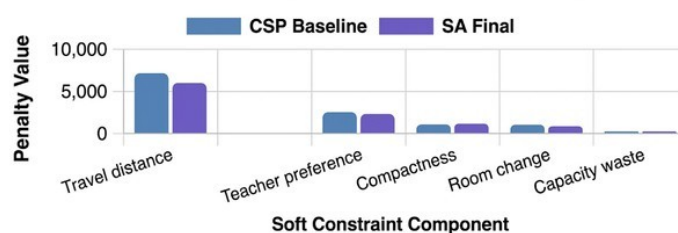
Figure 8 : Soft Cost Comparison: CSP vs SA vs Hill Climbing

The soft cost breakdown after SA optimization compared to the CSP baseline is shown in Table 4.

Soft constraint	CSP Baseline	After SA	Reduction (%)
Travel distance	10,772.50	1,322.50	87.7%'
Teacher time preference	2,140.00'	1,230.00'	42.5%'
Teacher room change	1,048.00	536.00'	48.9%'
Compactness	90.00'	120.00'	-33.3%'
Capacity waste	202.20'	202.20'	0%

Table 4 — Soft Cost Breakdown: CSP Baseline vs SA Final

Travel distance and teacher room changes showed the biggest improvements, which suggests that Simulated Annealing was effectively focusing on the most expensive parts of the schedule. The small increase in compactness is a normal trade-off: by moving sessions to reduce travel, the schedule sometimes ends up with small gaps in the day,



making it slightly less compact.

6.6 Room Utilization Analysis

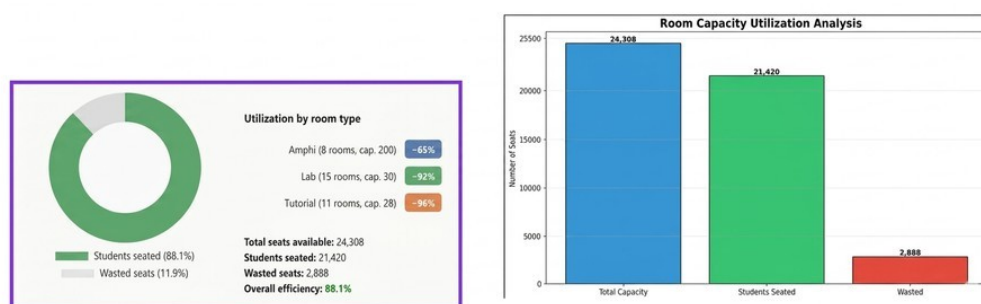


Figure 9

The final schedule achieves a room utilization efficiency of 88.1%, with 21,420 students placed out of a total capacity of 24,308 seats. This leaves 2,888 unused seats, or about 11.9% waste..

Most of this waste is not caused by the algorithm but by structural constraints in the room inventory. For example, amphitheatres can hold up to 200 students, while many lecture groups range between 112 and 168 students, which naturally creates mismatches in capacity. Because of this, some empty seats are unavoidable even in a well-optimized schedule.

The system already reduces this issue as much as possible by always assigning each session to the smallest room that can still fit the group. As a result, the remaining 11.9% can be seen as the lowest practical limit unless room sizes or group structures are changed.

7. Discussion

7.1 Interpretation of Results

The results show that the three-phase hybrid architecture successfully achieves its main goal: producing a fully feasible, conflict-free timetable for ENSIA while also improving schedule quality.

The CSP phase acts as a very strong feasibility engine. It manages to assign all 459 sessions in less than 0.2 seconds with zero backtracking, which confirms both the correctness of the constraint model and the effectiveness of the degree heuristic for variable ordering.

On the optimization side, Simulated Annealing clearly outperforms Hill Climbing. It achieves a 76.1% reduction in soft cost compared to 47.9% for Hill Climbing. This gap is explained by their exploration strategies: Hill Climbing accepts only improvements and, even with valid-shuffle restarts, tends to return to similar regions of the search space. Simulated Annealing escapes local optima more effectively through probabilistic acceptance of worse moves, allowing it to explore a broader solution space before converging.

Another important observation is the dominance of travel distance in the soft cost (75.6% of the CSP baseline and still the largest component after optimization). This reveals a structural property of the problem: when schedules are built mainly for feasibility, sessions often end up placed in different buildings without considering proximity. This is not a failure of the optimization process, but a consequence of fixing room assignments before time-slot optimization, which limits the ability to jointly optimize location and scheduling.

Capacity waste stays at 202 (0% reduction) for HC and SA because neither performs room swaps

SA stopped at iteration 17031 out of 200000 (only 8.5%) this is because the cooling rate 0.9995 was strong intentionally

7.2 Limitations

Several limitations of the current system should be noted.

First, the cost function is computationally expensive because it performs a full $O(D \times P \times U)$ scan of the timetable at every evaluation call — roughly 1,470 cell reads per iteration. Over the 17,031 SA iterations executed in this run, this amounts to approximately 25 million reads. Scaled to a full 200,000-iteration budget, the count would reach around 294 million reads. This is still manageable for the current dataset size, but it would become a serious bottleneck if the system were scaled to a larger institution or longer optimization runs. A delta evaluation approach, which only recomputes the cost for the modified parts of the schedule after each move or swap, would reduce this to around 10 reads per evaluation, leading to a potential 100–200× speedup.

Second, Hill Climbing does not use any memory mechanism. Because it relies only on random valid shuffles for restarts, it can repeatedly return to similar local optima instead of exploring new regions of the search space. Introducing a tabu list would likely improve its performance.

Additionally, soft constraint weights were set through domain judgment rather than systematic tuning; a sensitivity analysis of weight configurations represents a natural direction for future work

Finally, the move selection strategy is fully random and does not prioritize high-impact sessions. A more guided approach that focuses on sessions contributing most to the soft cost would likely improve convergence speed and overall solution quality.

7.3 Future Work

Several extensions could improve this system further.

The most impactful improvement would be implementing delta evaluation, which would allow much faster computation and enable longer optimization runs.

Another important direction would be to jointly optimize room assignment and scheduling instead of separating them. This would directly address the travel distance issue, although it would increase the complexity of the search space.

Adding a Tabu Search variant would also be valuable, as it would introduce a structured memory mechanism and provide a stronger alternative to random-restart strategies.

Finally, developing a simple web-based interface would make the system more practical for real use, allowing administrative users to interact with the timetable without needing to access the Jupyter notebook.

8. Conclusion

This project presented an automated course timetabling system for ENSIA, designed to handle a complex scheduling problem involving 459 sessions, 34 rooms, and 41 student groups across four academic years.

The proposed hybrid approach combined CSP for feasibility with Hill Climbing and Simulated Annealing for optimization. The CSP phase generated a fully feasible timetable in under 0.2 seconds without backtracking. Simulated Annealing achieved better optimization results than Hill Climbing, reducing the soft constraint cost by 76.1% compared to 47.9%, while preserving all hard constraints. The final timetable reached 88.1% room utilization efficiency.

Some limitations remain, such as full-scan cost evaluation and unguided move selection, which could be improved in future work without changing the overall system architecture.

References

- Müller, T., Rudová, H., & Müllerová, Z. (2007). ITC-2007: International Timetabling Competition. Retrieved from <https://www.cs.qub.ac.uk/ite2007/>
- Russell, S., & Norvig, P. (2020). Artificial intelligence: A modern approach (4th ed.). Pearson.
- Müller, T. (2012). UniTime: A comprehensive university timetabling system. Retrieved from <https://www.unitime.org>
- Pillay, N. (2016). A survey of school timetabling research. *Annals of Operations Research*, 218(1), 261–293. <https://doi.org/10.1007/s10479-013-1321-8>
- Babaei, H., Karimpour, J., & Hadidi, A. (2015). A survey of approaches for university course timetabling problem. *Computers & Industrial Engineering*, 86, 43–59. <https://doi.org/10.1016/j.cie.2014.11.010>
- Kristiansen, S., & Stidsen, T. R. (2013). A comprehensive study of educational timetabling — A survey. DTU Management Engineering Report.
- Abramson, D. (1991). Constructing school timetables using simulated annealing: Sequential and parallel algorithms. *Management Science*, 37(1), 98–113. <https://doi.org/10.1287/mnsc.37.1.98>
- Coloni, A., Dorigo, M., & Maniezzo, V. (1992). A genetic algorithm to solve the timetable problem. Polytechnico di Milano, Technical Report 90-060.

Appendix A:

```
print("❌ No CSP solution found. Try increasing timeout or max_backtracks.")

Loaded 459 sessions from 'sessions.json'

=====
CSP Backtracking Search
=====
Sessions      : 459
Grid          : 5d x 6p = 30 slots
Conflict pairs: 9476
Domain sizes  : min=24, avg=24.0
AC-3 enabled  : False
Timeout       : 300.0s
=====

SOLUTION FOUND
Time      : 0.15s
Nodes     : 459
Backtracks: 0
Assigned  : 459/459 sessions

✅ CSP solution saved to 'schedule_output.json' (459 sessions)
```

```
print("\n✅ Data loaded successfully.")

Loading timetable data...
TimetableContext: 5d x 6p, 459 sessions, 34 rooms, 8 sections, 41 groups

Sessions : 459
Rooms    : 34
Sections : 8
Groups   : 41
Days     : 5
Periods  : 6

Session types : {'lecture': 62, 'tuto': 203, 'lab': 194}
Room types    : {'amphi': 8, 'lab': 15, 'tuto': 11}

✅ Data loaded successfully.
```

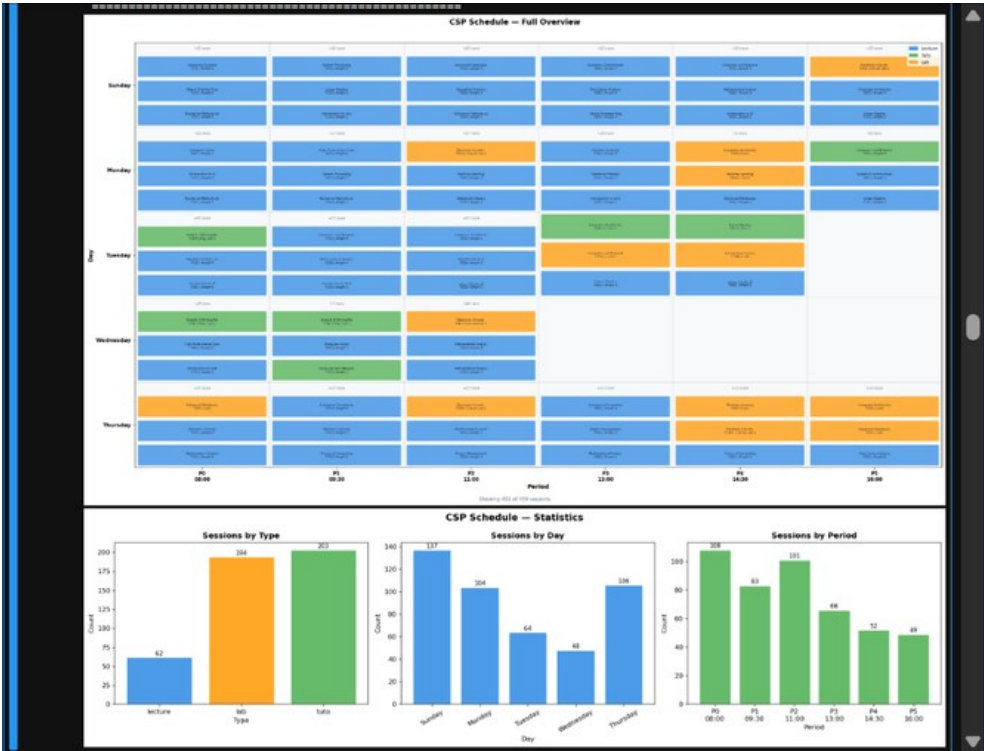
"Data loading output confirming successful loading of 459 sessions, 34 rooms, 8 sections, and 41 groups into the TimetableContext."

```
=====
HARD CONSTRAINT VALIDATION - CSP OUTPUT
=====
✅ SCHEDULE VALID
459 sessions checked - no hard constraint violations found.

Sessions checked : 459
Total violations : 0
Result           : PASS ✅

Soft cost breakdown (CSP):
capacity_waste      : 202.2
teacher_room_change : 1048.0
travel_distance     : 10772.5
compactness         : 90.0
teacher_preference   : 2140.0
total               : 14252.7
```

CSP Schedule — Full Overview and statistics, showing session distribution by type, day, and period across all 459 sessions from the initial feasible solution



CSP Schedule Full Overview and Statistics: Color-coded full weekly timetable generated by the CSP solver showing all 459 sessions across 5 days and 6 periods, alongside session distribution statistics by type (62 lectures, 194 labs, 203 tutorials), by day, and by period.



CSP Schedule Section Viewer: Interactive weekly timetable viewer showing the CSP-generated schedule for sections Y1S1 and Y1S2, with sessions color-coded by type (blue = lecture, green = tutorial, orange = lab) and displaying room and group information for each session across the five-day week

```
SA starting from CSP schedule: 459 sessions

Initial soft cost breakdown (CSP + SA input):
capacity_waste      : 202.2
teacher_room_change : 1048.0
travel_distance     : 10772.5
compactness         : 90.0
teacher_preference   : 2140.0
total               : 14252.7

=====
Simulated Annealing - Soft Cost Optimisation
=====
Sessions      : 459
T_start       : 500.0
T_end         : 0.1
Cooling        : 0.9995
Max iter      : 200,000
Seed          : 42
=====
Initial soft cost : 14252.7000
```

Simulated Annealing Initialization: SA configuration and CSP baseline soft cost breakdown used as the starting point for optimization. The initial soft cost is 14,252.70, with travel distance (10,772.5) as the dominant component. SA parameters: T_start = 500, cooling rate = 0.9995, maximum iterations = 200,000.

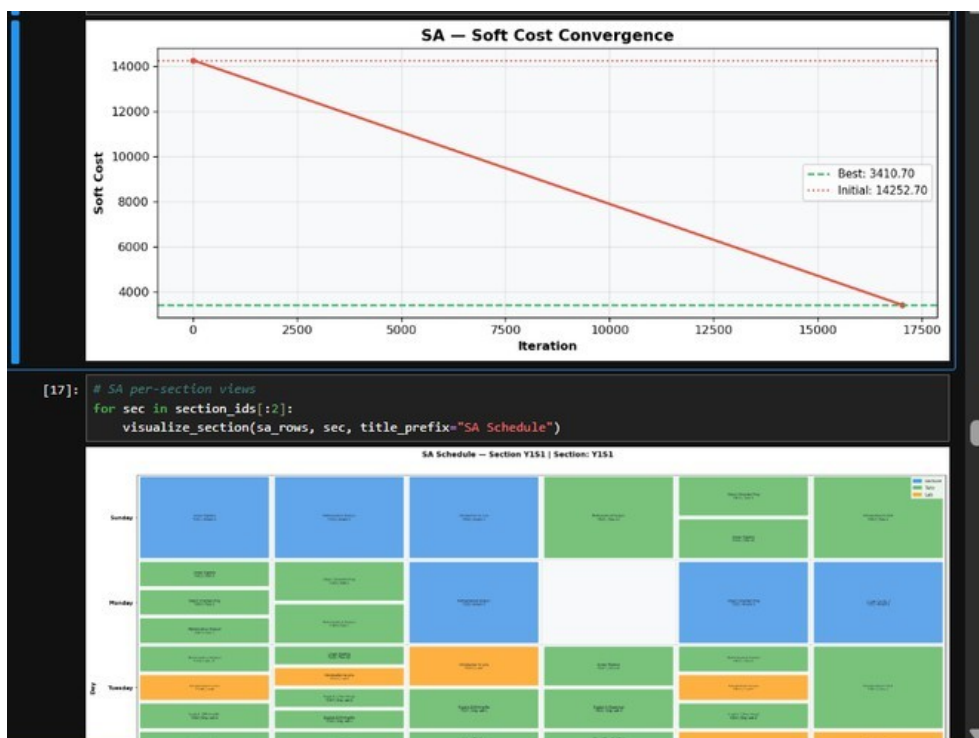
```
=====
HARD CONSTRAINT VALIDATION - SA OUTPUT
=====
[✓] SCHEDULE VALID
459 sessions checked - no hard constraint violations found.

Sessions checked : 459
Total violations : 0
Result           : PASS [✓]
```

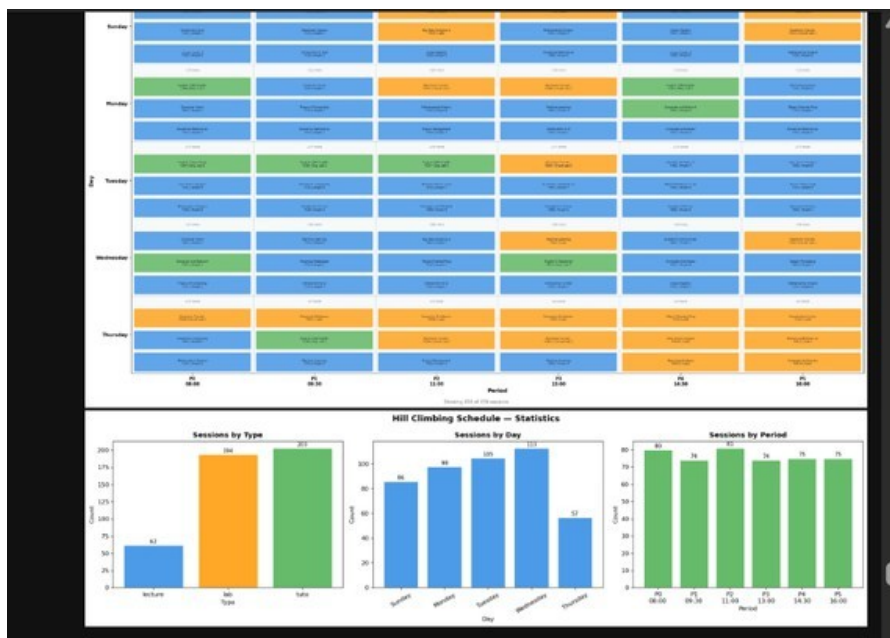
Simulated Annealing Initialization: Hard constraint Validation



SA Schedule Full Overview & Statistics: Color-coded full weekly timetable generated by Simulated Annealing showing all 459 sessions across 5 days and 6 periods, alongside session distribution statistics by type (62 lectures, 184 labs, 203 tutorials), by day, and by period.



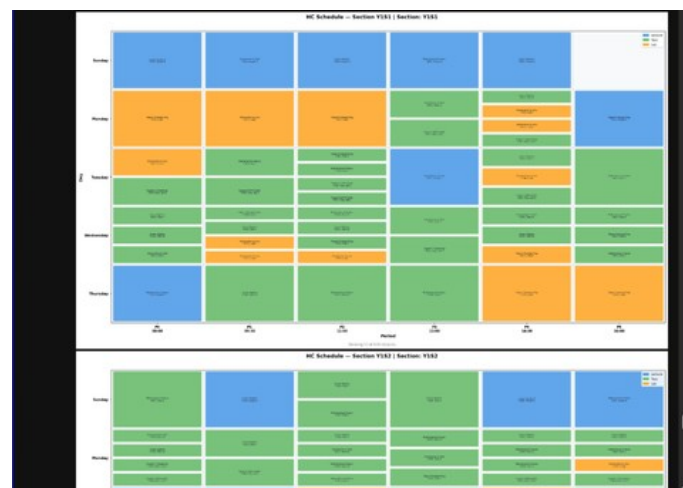
SA Soft Cost Convergence & Section Viewer: SA convergence curve showing a steady reduction from the CSP baseline of 14,252.70 down to a best cost of 3,410.70 over 17,031 iterations, representing a 76.1% improvement. Below, the SA timetable viewer displays the optimized weekly schedule for section Y1S1.



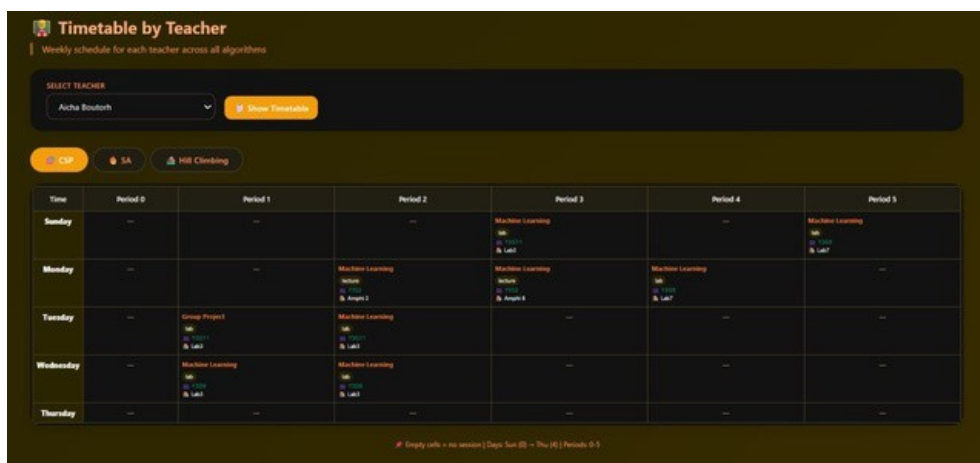
Hill Climbing Schedule Full Overview & Statistics: Color-coded full weekly timetable generated by Hill Climbing showing all 459 sessions across 5 days and 6 periods, alongside session distribution statistics by type (62 lectures, 194 labs, 203 tutorials), by day, and by period.



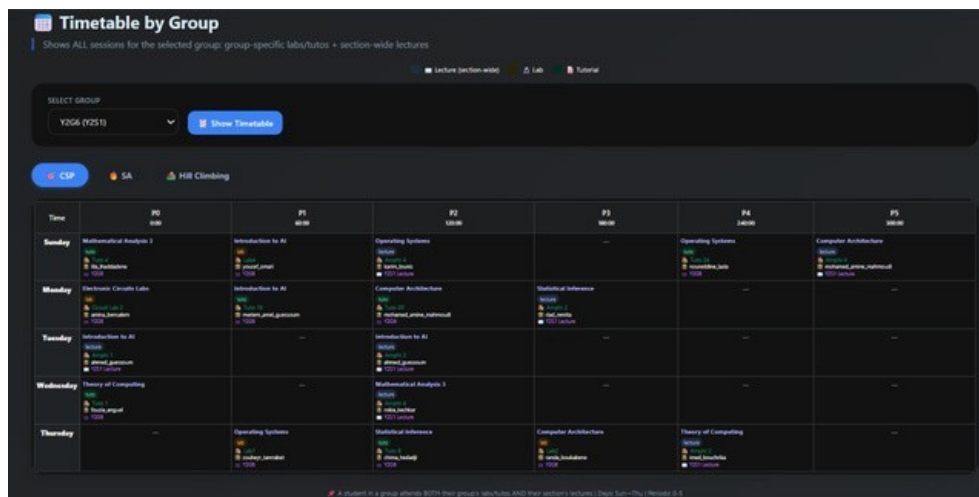
— Hill Climbing Best Cost per Restart: Convergence curve showing the best soft cost achieved at each restart, dropping rapidly from the CSP baseline of 14,252.70 in the first few restarts before stabilizing around 7,419.20 after restart 28, with no further improvement over the remaining restarts — illustrating the typical local optima trapping behavior of Hill Climbing.



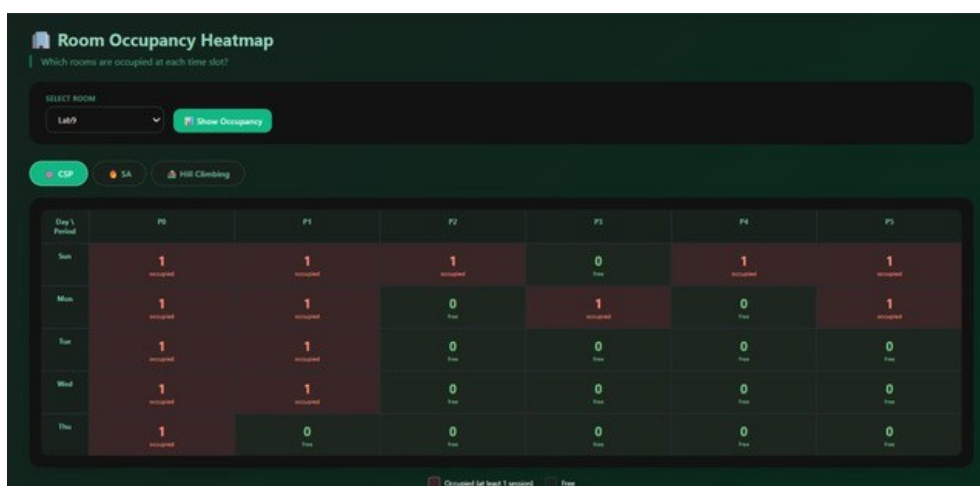
— HC Schedule Section Viewer: Interactive weekly timetable viewer showing the Hill Climbing optimized schedule for sections Y1S1 and Y1S2, with sessions color-coded by type (blue = lecture, green = tutorial, orange = lab) and displaying room and group information for each session across the five-day week.



Timetable by Teacher Viewer: Interactive web-based schedule viewer displaying the weekly timetable for a selected teacher (here: Aicha Boutorh) across all three algorithm outputs (CSP, SA, Hill Climbing), showing session type, assigned group, and room for each time slot across the five-day week.



Timetable by Group Viewer: Interactive weekly schedule viewer showing all sessions for a selected student group (here: Y2G6 from section Y2S1), combining group-specific labs and tutorials with section-wide lectures, color-coded by session type (blue = lecture, orange = lab, brown = tutorial). The viewer supports switching between CSP, SA, and Hill Climbing outputs.



Room Occupancy Heatmap: Interactive viewer showing the weekly occupancy status of a selected room (here: Lab9) across all 5 days and 6 periods, with red cells indicating occupied slots and dark cells indicating free slots. The viewer supports switching between CSP, SA, and Hill Climbing outputs to compare room usage across algorithms.

Appendix B

Full Name	Group	Report Contribution	Code Contribution
Hammoutene Saliha	G05	Abstract;Problem Solving techniques	Simulated annealing algorithm/ work on comparison graphs
Tabti Lydia	G05	Results and Analysis	Hill climbing algorithm/ visualization (timetables)
Djeriba Dhoha	G06	Conclusion + Appendix A +Appendix B+State of the Art	CSP algorithm/ dataset_structure and data extraction
Benzine Amina	G06	Problem Solving Techniques+Introduction	Simulated annealing algorithm/ soft constraints
Merzouk Nourimane	G06	Application Design+Discussion	Hill Climbing/work on hard constraints used in CSP algorithm
Bendjemaa Imane (leader)	G06	Dataset Building+References	CSP algorithm/ dataset_structure and data extraction